9.2 stringr: Regular Expressions

A pattern is a regular expression. Type `"\\d"` for a digit and `"\\w"` for a word character.

1. A pattern is a regular expression

Note 9.1 used literal text. A **regular expression** describes a set of strings. `stringr` (through `stringi`) uses ICU regex. Type the pattern as an R string: `"\\d"` for a digit, `"\\w"` for a word character, and `"\\."` for a real dot.

`words` is 980 common English words that come with stringr. Use it for the examples.

```
library(tidyverse)

str_view(words, "ie")
```

```
##   [7] │ ach<ie>ve
##  [86] │ bel<ie>ve
## [111] │ br<ie>f
## [158] │ cl<ie>nt
## [228] │ d<ie>
## [292] │ exper<ie>nce
## [310] │ f<ie>ld
## [342] │ fr<ie>nd
## [472] │ l<ie>
## [620] │ p<ie>ce
## [670] │ qu<ie>t
## [726] │ sc<ie>nce
## [781] │ soc<ie>ty
## [872] │ t<ie>
## [918] │ v<ie>w
```

`str_view()` shows the match. It is the fastest way to test a pattern.

---

2. The language, in pieces

`.` matches any single character (except a newline). Escape it when you want a real dot: `"\\."`.

```
str_view(c("abc", "a.c", "ac"), "a.c")
```

```
## [1] │ <abc>
## [2] │ <a.c>
```

```
str_view(c("abc", "a.c", "ac"), "a\\.")
```

```
## [2] │ <a.>c
```

`^` is the start of the string. `$` is the end.

```
str_view(words, "^a")
```

```
##  [1] │ <a>
##  [2] │ <a>ble
##  [3] │ <a>bout
##  [4] │ <a>bsolute
##  [5] │ <a>ccept
##  [6] │ <a>ccount
##  [7] │ <a>chieve
##  [8] │ <a>cross
##  [9] │ <a>ct
## [10] │ <a>ctive
## [11] │ <a>ctual
## [12] │ <a>dd
## [13] │ <a>ddress
## [14] │ <a>dmit
## [15] │ <a>dvertise
## [16] │ <a>ffect
## [17] │ <a>fford
## [18] │ <a>fter
## [19] │ <a>fternoon
## [20] │ <a>gain
## ... and 45 more
```

```
str_view(words, "x$")
```

```
## [108] │ bo<x>
## [747] │ se<x>
## [772] │ si<x>
## [841] │ ta<x>
```

```
str_view(words, "^apple$")
```

A **character class** matches one character from a set: `[abc]`, `[a-z]`, `[0-9]`. `[^aeiou]` is "not a vowel." `[^ ]` is "not a space."

```
str_view(words, "^[aeiou]")
```

```
##  [1] │ <a>
##  [2] │ <a>ble
##  [3] │ <a>bout
##  [4] │ <a>bsolute
##  [5] │ <a>ccept
##  [6] │ <a>ccount
##  [7] │ <a>chieve
##  [8] │ <a>cross
##  [9] │ <a>ct
## [10] │ <a>ctive
## [11] │ <a>ctual
## [12] │ <a>dd
## [13] │ <a>ddress
## [14] │ <a>dmit
## [15] │ <a>dvertise
## [16] │ <a>ffect
## [17] │ <a>fford
## [18] │ <a>fter
## [19] │ <a>fternoon
## [20] │ <a>gain
## ... and 155 more
```

```
str_view(c("a b", "ab"), "[^ ]")
```

```
## [1] │ <a> <b>
## [2] │ <a><b>
```

`(?i)` at the front makes the rest of the pattern case-insensitive.

```
str_detect(c("Apple", "BANANA", "pear"), "(?i)^a")
```

```
## [1]  TRUE FALSE FALSE
```

Repetition sits after the thing it repeats:

- `?` — 0 or 1
- `+` — 1 or more
- `*` — 0 or more
- `{n}` — exactly n
- `{n,}` — n or more
- `{n,m}` — between n and m

```
str_view(words, "^a.{3}$")
```

```
##  [2] │ <able>
## [33] │ <also>
## [49] │ <area>
## [64] │ <away>
```

```
str_view(c("color", "colour"), "colou?r")
```

```
## [1] │ <color>
## [2] │ <colour>
```

Parentheses make a **group**. `\\1` is "whatever group 1 matched." Note 4.2 used `matches("(.)\\1")` in `select()`. That is a repeated character.

```
str_view(words, "(.)\\1")
```

```
##  [5] │ a<cc>ept
##  [6] │ a<cc>ount
##  [8] │ acro<ss>
## [12] │ a<dd>
## [13] │ a<dd>re<ss>
## [16] │ a<ff>ect
## [17] │ a<ff>ord
## [19] │ aftern<oo>n
## [25] │ agr<ee>
## [27] │ a<ll>
## [28] │ a<ll>ow
## [43] │ a<pp>arent
## [44] │ a<pp>ear
## [45] │ a<pp>ly
## [46] │ a<pp>oint
## [47] │ a<pp>roach
## [48] │ a<pp>ropriate
## [53] │ a<rr>ange
## [57] │ a<ss>ociate
## [58] │ a<ss>ume
## ... and 137 more
```

```
str_replace("apple", "(.)(.)", "\\2\\1")
```

```
## [1] "paple"
```

---

3. Capturing groups, non-capturing groups, and lookaheads

Note 9.5 is the full lesson, with more cases and practice questions. The examples here are the ones to work in class.

Capturing groups `(...)`

A capturing group saves the text it matched. `str_match()` puts the whole match in column 1, then one column for each group, from left to right.

```
str_match("John123", "([A-Za-z]+)([0-9]+)")
```

```
     [,1]      [,2]   [,3]
[1,] "John123" "John" "123"
```

Column 2 is the letters. Column 3 is the digits.

```
str_match(c("Alice45", "Bob789", "Sam"), "([A-Za-z]+)([0-9]+)")
```

```
     [,1]      [,2]    [,3]
[1,] "Alice45" "Alice" "45"
[2,] "Bob789"  "Bob"   "789"
[3,] NA        NA      NA
```

`"Sam"` has letters but no digits, so that row is missing.

```
str_match("2026-10-28", "(\\d{4})-(\\d{2})-(\\d{2})")
```

```
     [,1]         [,2]   [,3] [,4]
[1,] "2026-10-28" "2026" "10" "28"
```

The three groups are the year, the month, and the day.

Non-capturing groups `(?:...)`

`(?:...)` matches, and it does not get a column. It also does not count when you number `\\1` and `\\2`.

```
str_match("John123", "(?:[A-Za-z]+)([0-9]+)")
str_match("John123", "([A-Za-z]+)(?:[0-9]+)")
```

```
     [,1]      [,2]
[1,] "John123" "123"
```

```
     [,1]      [,2]
[1,] "John123" "John"
```

The first pattern keeps only the digits. The second keeps only the name. The whole match is still `"John123"`.

```
str_detect(c("gray", "grey", "green"), "gr(?:a|e)y")
```

```
[1]  TRUE  TRUE FALSE
```

`(?:a|e)` is `a` or `e`. `gray` and `grey` match. `green` does not. `str_detect()` returns true or false, so there is no column to capture.

Positive lookahead `(?=...)`

`(?=...)` checks what comes next. It does not take those characters, and it does not move past them.

```
str_extract(c("cat123", "catABC", "dog9"), "cat(?=[0-9])")
```

```
[1] "cat" NA    NA
```

`"cat123"` is `cat` followed by a digit, so the match is `"cat"`. `"catABC"` is `cat` followed by a letter, so it fails. `"dog9"` has no `cat`.

```
str_extract(c("50 dollars", "50 cents", "7 dollars"), "\\d+(?= dollars)")
```

```
[1] "50" NA   "7"
```

The match is the number. `(?= dollars)` only checks that the word `dollars` comes next. `"50 cents"` fails that check.

```
str_extract(c("notes.pdf", "notes.docx", "plot.pdf"), "\\w+(?=\\.pdf)")
```

```
[1] "notes" NA      "plot"
```

The match is the file name. `(?=\\.pdf)` checks for a literal `.pdf` and does not take it. `"notes.docx"` fails that check.

A pattern that takes the next character keeps it:

```
str_extract("cat123", "cat[0-9]")
str_extract("cat123", "cat(?=[0-9])")
```

```
[1] "cat1"
```

```
[1] "cat"
```

The first result is `"cat1"`. The second is `"cat"`.

Negative lookahead `(?!...)`

`(?!...)` checks that a pattern does not come next. It does not take any characters.

```
str_detect(c("cat", "catalog", "dog", "bird"), "^(?!cat)")
```

```
[1] FALSE FALSE  TRUE  TRUE
```

`FALSE` means the string starts with `cat`. `"catalog"` fails for the same reason: it starts with `cat`. `"dog"` and `"bird"` do not.

```
str_extract(c("cat", "cats", "dog"), "cat(?!s)")
```

```
[1] "cat" NA    NA
```

`"cat"` is not followed by `s`, so it matches. `"cats"` is `cat` followed by `s`, so the check fails. `"dog"` has no `cat`.

```
str_detect(c("all good", "an error here", "no problem"), "^(?!.*error).*$")
```

```
[1]  TRUE FALSE  TRUE
```

`(?!.*error)` fails when `error` appears anywhere later in the string. The other two strings match.

A group, capturing or not, is part of the match. A lookahead only tests a condition.

---

4. Detect, count, extract, split, replace

All of these take the vector first, then the pattern.

```
x <- c("apple", "banana", "pear", "mango")

str_detect(x, "a")
```

```
## [1] TRUE TRUE TRUE TRUE
```

```
x[str_detect(x, "^p")]
```

```
## [1] "pear"
```

```
str_subset(x, "^p")
```

```
## [1] "pear"
```

```
str_count(x, "a")
```

```
## [1] 1 3 1 1
```

```
str_count("abababa", "aba")
```

```
## [1] 2
```

```
str_extract(x, "[aeiou]")
```

```
## [1] "a" "a" "e" "a"
```

```
str_extract_all(x, "[aeiou]")
```

```
## [[1]]
## [1] "a" "e"
##
## [[2]]
## [1] "a" "a" "a"
##
## [[3]]
## [1] "e" "a"
##
## [[4]]
## [1] "a" "o"
```

```
str_split("a-b-c", "-")
```

```
## [[1]]
## [1] "a" "b" "c"
```

```
str_replace(x, "[aeiou]", "-")
```

```
## [1] "-pple"  "b-nana" "p-ar"   "m-ngo"
```

```
str_replace_all(x, "[aeiou]", "-")
```

```
## [1] "-ppl-"  "b-n-n-" "p--r"   "m-ng-"
```

`str_count()` counts **non-overlapping** matches. `"aba"` in `"abababa"` is 2, not 3.

`str_extract()` is the first match. `str_extract_all()` is every match, as a list.

`str_subset(x, pattern)` is `x[str_detect(x, pattern)]`.

---

5. `separate()` with a regex

`tidyr::separate()` splits one column into several. `sep` can be a regex, not only a fixed character.

```
tibble(x = c("a-1", "b.2", "c 3")) |>
  separate(x, into = c("letter", "num"), sep = "[^[:alnum:]]")
```

```
## # A tibble: 3 × 2
##   letter num
##   <chr>  <chr>
## 1 a      1
## 2 b      2
## 3 c      3
```

`[^[:alnum:]]` is any character that is not a letter or a digit: a hyphen, a dot, or a space.

---

6. Practice

Use `words`. Print the matches (or `str_view()` them).

1. Four-letter words that start with `a`.
2. Words that start with a vowel.
3. Words that end in `ed` but not `eed`.
4. Words that start with three consonants.